

第7课 模块化方法

严禁复制

汇报人：北京交通大学软件学院

什么是模块化？

- 在软件开发过程中，将系统分解成多个模块；
- 并且每个模块都能独立开发。





大泥团架构

严禁复制

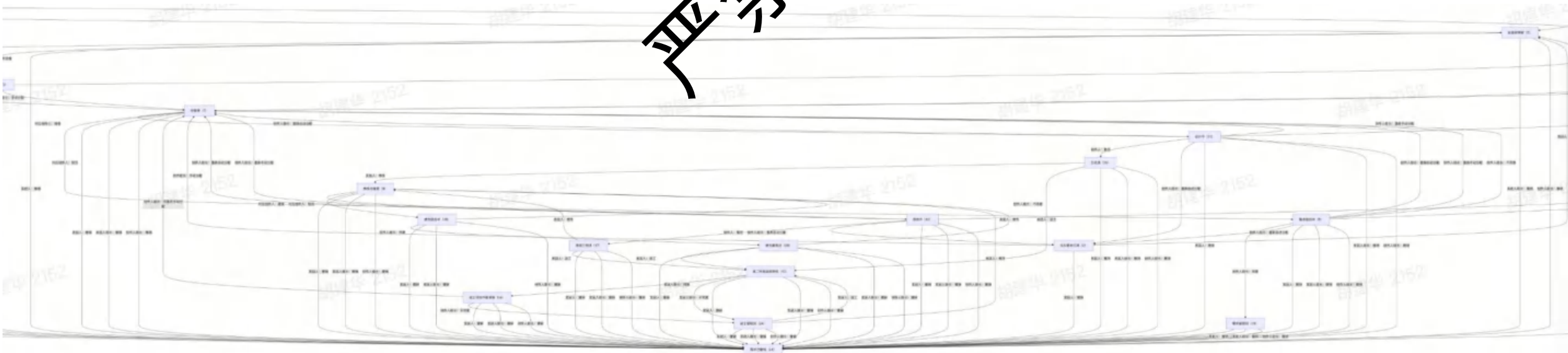
第一章



大泥团架构

- Big Ball of Mud
- 元素
 - 执行各种任务的元素的交织在一起
- 约束
 - 没有任何约束

严禁复制



大泥团架构-质量属性

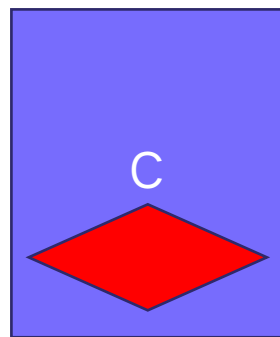
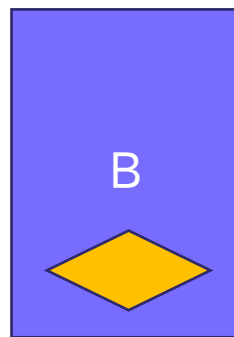
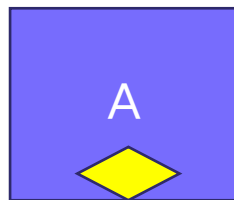
- 快速上市
 - 快速完成：一次性完成系统，不需要修改和升级；
 - 根据需要解决问题：增加或修改功能；
- 低成本
 - 短期来说是成本低廉的解决方案；

严禁复制



大泥团架构-问题

- 难维护
 - 运维，扩容，升级基础组件
- 难于修改
 - 刚开始时很灵活，过了一段时间后，很难修改
- 惯性驱动
 - 一旦一个系统成为大泥团架构，很难作出改变
 - 只有熟悉系统的开发人员才知道改哪里
 - 对代码有要求的开发者拒绝参与
- 技术债



大泥团架构-适用场景

- 可抛弃的代码
 - 快速修复一个小问题（内存泄漏）
 - 快速原型证明概念（用LLM回答问题）
- Copy&Paste的复用
- 任务简单且需求稳定
 - redis sdk

严禁复制





模块化

第二章

严禁复制



模块的定义

- 模块

- 软件的一部分，提供一些职责（不是功能）
- 在构建过程中关注（在运行过程中不关注）
- 分为接口和实现两部分

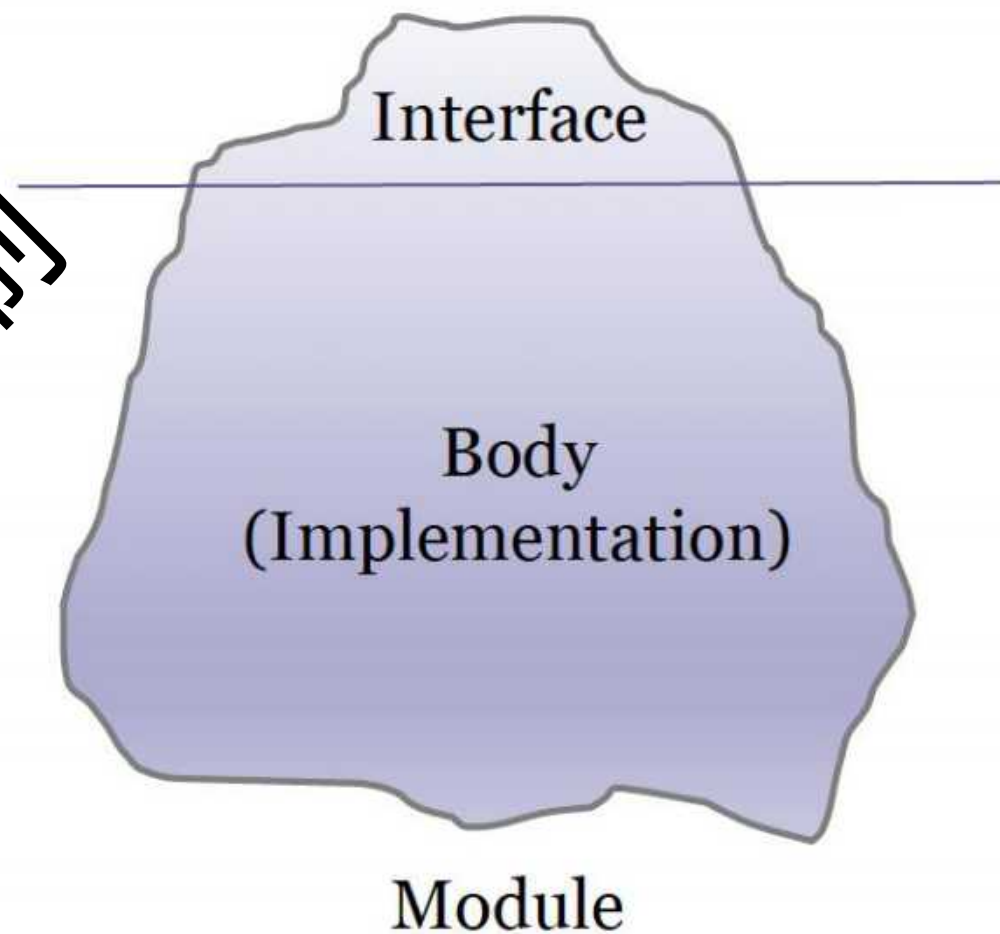
- 接口

- 描述接口的职责
- 契约

- 实现

- 实现接口所包含的职责

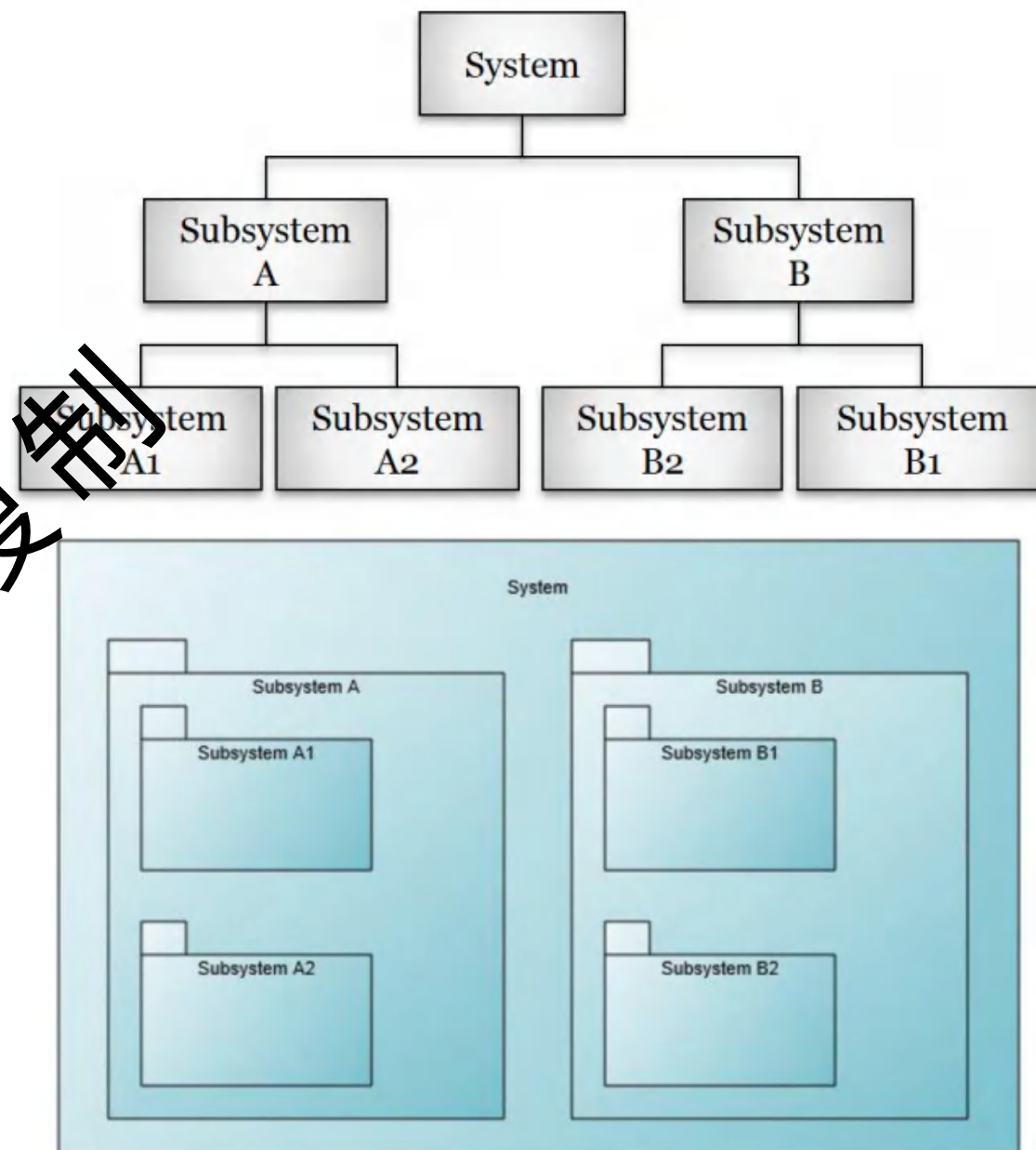
严禁复制



java 的class，python的py文件，是最细粒度的模块。

模块的划分方法

- 关系: is-part-of
- 约束:
 - 不能形成循环 (cycle)
 - 通常情况, 一个模块只能有一个上级模块
- 表达方式
 - 树形图
 - 模块图



模块化支持的质量属性

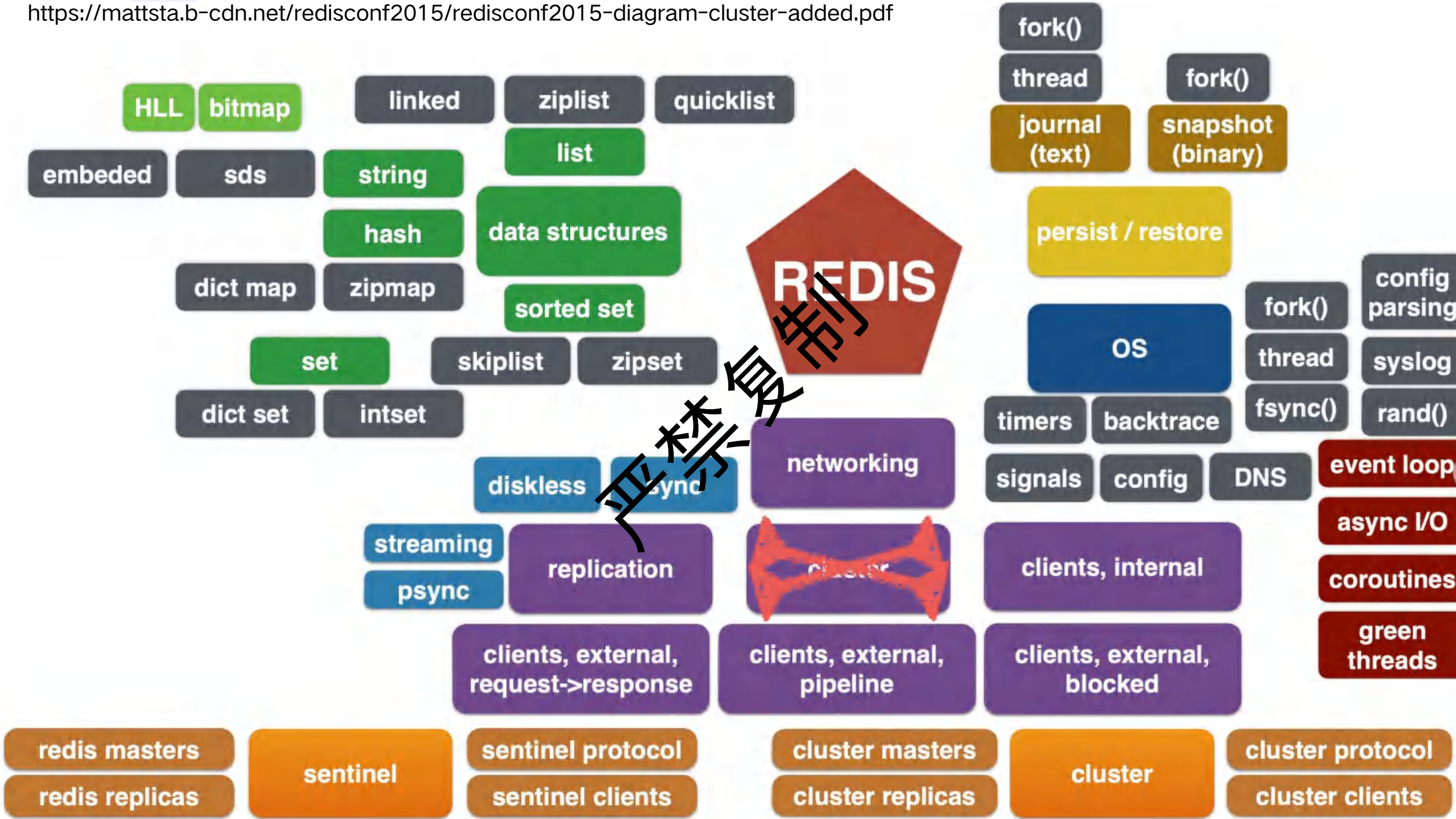
- 沟通交流
 - 很容易讲清楚系统的核心功能
- 可维护性
 - 容易变更和扩展
 - 本地化
- 简单性（容易理解）
 - 只需要理解接口，不需要理解实现细节
- 可复用性（提高效率）
 - 模块可以运行在不同的系统中（机器学习模块，数据结构模块）
- 独立性
 - 不同模块可以由不同团队开发（大泥团只能由一个团队开发）

禁止复制

模块化的挑战

- 不合理的模块划分增加复杂性
 - 方案1更简洁，更容易排查错误
- 依赖管理
 - 模块化可能限制发展（不支持流式）
- 团队组织结构
 - 模块划分影响团队组织结构
- 决定是自研还是购买
 - COTS（商业化）/FOSS（开源）

严禁复制





模块化战术

严禁复制

第三章



模块化战术

1. SOLID原则

2. 提高内聚

3. 降低耦合

4. 消除共生性

5. 模块集成

- Postel定律, Demeter 定律, 流畅接口模式

6. GRASP模式

7. 其它战术

禁止复制

The background features a minimalist illustration of a traditional Chinese landscape. In the upper left, several birds are depicted in flight. The right side of the image is dominated by stylized, layered mountain peaks in shades of light gray. At the base of these mountains, a small pavilion with a traditional Chinese roof stands on a dark, silhouetted shoreline. The overall aesthetic is clean and modern, with a focus on natural elements.

SOLID

第四章

严禁复制

Solid原则

- Solid原则可以用于不同层级的模块
 - SRP（单一职责）
 - OCP（开闭原则）
 - LSP（里氏替换原则）
 - ISP（接口分离原则）
 - DIP（依赖翻转原则）

严禁复制

SRP单一职责

- 定义:
 - 一个类应该只有一个引起它变化的原因。
 - 换句话说，一个类应该只负责一个功能或任务。
- 含义:
 - 如果一个类承担了太多的责任，那么修改其中一个责任可能会影响到其他的责任。
 - 这会导致代码变得脆弱、难以理解和测试。
- 好处:
 - 高内聚: 将相关的职责集中在一个类中，提高代码的内聚性。
 - 低耦合: 减少类之间的依赖关系，降低代码的耦合性。
 - 易于维护: 只需要修改相关的类，而不会影响到其他的类。
 - 易于测试: 可以更容易地对每个类进行单元测试，因为每个类只负责一个功能。

(S)单一职责

- 一个模块**必须**只拥有一个职责（不要纠结什么是职责）
 - 职责可以划分为更细的职责，但是不能包含**同级别**的其它模块负责的职责
 - 商品订单 VS 订单结算
 - 单一职责 = 只有一个变化的理由（失败了只有一个原因）
 - 不能超过一个理由
 - 增加debug复杂度
 - 增加耦合



严禁复制

VS



单一职责

Order 类包含了与订单相关的多个责任：

1. 订单的创建和管理（例如
add_item, calculate_total）
2. 数据持久化（例如
save_to_database）
3. 发送确认邮件（例如
send_confirmation_email）

```
class Order:
    def __init__(self, order_id, user, items):
        self.order_id = order_id
        self.user = user
        self.items = items
        self.status = 'created'

    def add_item(self, item):
        self.items.append(item)

    def calculate_total(self):
        return sum(item.price for item in self.items)

    def save_to_database(self):
        # 这里是保存订单到数据库的代码
        pass

    def send_confirmation_email(self):
        # 这里是发送订单确认邮件的代码
        pass
```

单一职责的例子

将订单拆分为多个类：

1. Order 类只负责订单管理的功能。
2. OrderRepository 类负责数据持久化。
3. EmailService 类负责发送邮件。

```
class Order:
    def __init__(self, order_id, user, items):
        self.order_id = order_id
        self.user = user
        self.items = items
        self.status = 'created'

    def add_item(self, item):
        self.items.append(item)

    def calculate_total(self):
        return sum(item.price for item in self.items)

class OrderRepository:
    def save(self, order):
        # 负责保存订单到数据库
        pass

class EmailService:
    def send_confirmation(self, order):
        # 负责发送订单确认邮件
        pass
```

开闭原则 (Open/Closed Principle, OCP)

- 定义:

- 软件实体（类、模块、函数等）应该是可扩展的，但是不可修改的。

- 含义:

- 应该允许通过增加新的代码来扩展一个类的功能，而不是修改已有的代码。

- 好处:

- 稳定性: 避免修改已有的代码，减少引入错误的风险。
- 可扩展性: 可以方便地添加新的功能，而不需要修改已有的代码。
- 可维护性: 更容易维护和升级代码，因为不需要修改已有的代码。

(O) 开 (open) 闭 (closed) 原则

- Open to 扩展:
 - 模块可以支持新需求
 - 变更或者适配模块的行为
- Closed to 修改:
 - 实现变化但不需要修改模块
- 原因:
 - 模块修改会带来风险
 - 第三方模块不能修改

例子：修改排序算法

```
List<Person> people = new ArrayList<>();
people.add(new Person("Alice", 30));
// 使用自定义 Comparator 进行排序（按年龄升序）

Comparator<Person> ageComparator =
    new Comparator<Person>() {
        public int compare(Person p1, Person p2) {
            return Integer.compare(p1.getAge(), p2.getAge());
        }
    };
// 排序算法不可变更
Collections.sort(people, ageComparator);
```

例1：调用方不可变更

写一个儿童画板软件，可以在上面画各种图形：矩形，圆形，三角形。

```
class ShapeDrawerBad2 {  
    public void drawCircle() {  
        //省略;  
    }  
    public void drawSquare() {  
  
    }  
    public void drawTriangle() {  
        //省略;  
    }  
}
```

```
// 抽象类 Shape  
abstract class Shape {  
    abstract void draw();  
}
```

// 具体子类

```
class Circle extends Shape {  
    void draw() { }  
}  
class Square extends Shape {  
    void draw() { }  
}  
class Triangle extends Shape {  
    void draw() { }  
}
```

只需要增加
子类，不需
要修改调用
方代码。

严禁复制

有哪些常见的扩展点



算法的切换:

- 排序算法: 从冒泡排序切换到快速排序。
- 搜索算法: 从线性搜索切换到二分搜索。
- 机器学习: 从线性回归切换到逻辑回归。



数据存储的变化:

- 数据库类型: 关系型数据库切换到 NoSQL
- 数据结构: 表结构的修改、字段的增删。
- 数据来源: 从本地文件改为从 API 获取



外部服务 (API) 的变化:

- API 版本的升级: 参数的变化。
- API 服务的切换: 第三方服务切换。
- API 协议的切换: 从 RESTful API 切换到 GraphQL API。



业务流程变化:

- 业务流程: 增加处理步骤。
- 业务规则: 修改判断条件。
- 工作流: 步骤调整、审批流程的变化。

里氏替换原则 (Liskov Substitution Principle, LSP)

- 定义:
 - 子类型必须能够替换掉它们的父类型，并且程序的行为不会受到影响。
- 含义:
 - 任何使用父类对象的地方，都应该能够使用子类对象来替换，而不会导致程序出错。
- 好处:
 - 代码重用: 子类可以继承父类的代码，并进行扩展或修改，而不需要重新编写代码。
 - 多态性: 可以使用父类类型的变量来引用子类对象，从而实现多态性。
 - 可维护性: 更容易维护和升级代码，因为可以放心地使用子类对象来替换父类对象。

```
def func( parent):
```

```
    xxx
```

```
    xxx
```

```
    xxx
```

```
func(son) // son是parent的子类
```

(L) 里氏可替换原则

- 要求：子模块必须遵守上级模块的契约

- 如果Parent是Son的上级

- Parent的一切属性，Son也必须具备
 - Son可以增加新

- 这样在用到Son的地方，才能用Parent替换

- 常见错误

- 子类修改了基类的必要^{必要}的行为

- 错误：父类加密存储了，子类没有

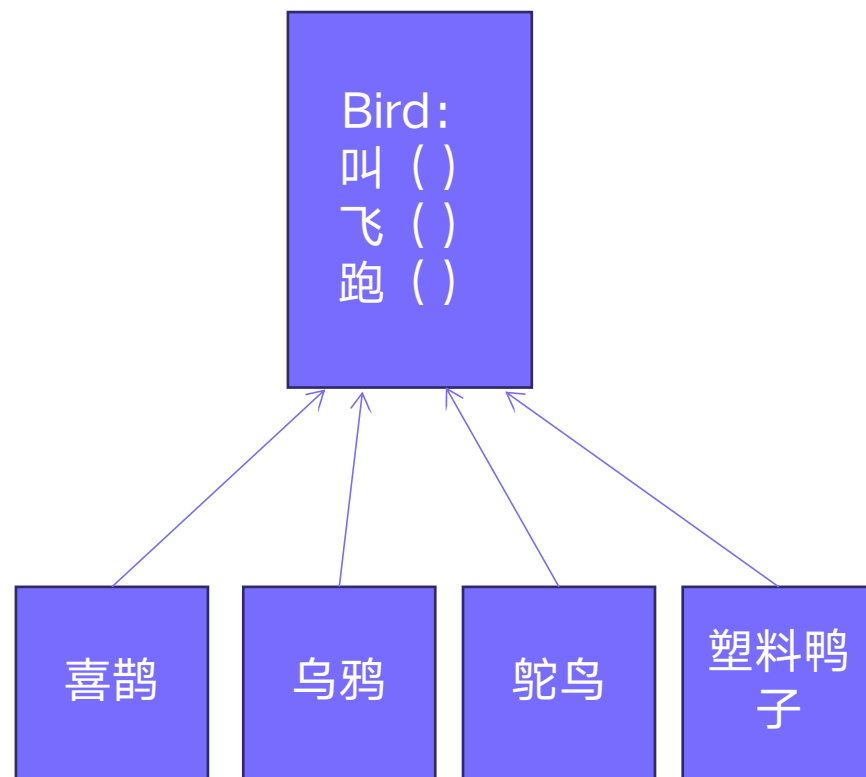
- 父类的定义了不必要^{不必要}的方法

- 错误：子类被强制支持特定的行为

- 如何改正：

- 定义合适的基类

严禁复制



LSP: 子类不遵守父类约定导致错误

```
class Bird:
    def fly(self, point1, point2):
        """从 point1 飞到 point2"""
        print(f"从 {point1} 飞到 {point2}")

    def run(self, point1, point2):
        """从 point1 跑到 point2"""
        print(f"从 {point1} 跑到 {point2}")

class Ostrich(Bird):
    def fly(self, point1, point2):
        """鸵鸟不会飞, 所以抛出异常"""
        raise Exception("鸵鸟不会飞!")

    def run(self, point1, point2):
        print(f"鸵鸟从 {point1} 跑到 {point2}")

class Eagle(Bird):
    def fly(self, point1, point2):
        print(f"老鹰从 {point1} 飞到 {point2}")

    def run(self, point1, point2):
        print(f"老鹰从 {point1} 跑到 {point2}")
```

```
class Game:
    def play(self, bird, point1, point2, is_road_connected):
        """根据道路是否连接, 决定鸟是跑还是飞"""
        if is_road_connected:
            bird.run(point1, point2)
        else:
            bird.fly(point1, point2)
```

输出: 老鹰从 A 跑到 B
输出: 老鹰从 A 飞到 B

输出: 鸵鸟从 C 跑到 D

else) # 抛出异常: 鸵鸟不会飞!

接口隔离原则 (Interface Segregation Principle, ISP)

- 定义:
 - 客户端不应该被迫依赖于它们不使用的接口。
- 含义:
 - 应该将大的接口拆分成更小的、更具体的接口，使得客户端只需要依赖于它们需要的接口。
- 好处:
 - 解耦: 减少类之间的依赖关系，降低代码的耦合性。
 - 灵活性: 可以更容易地修改和扩展接口，而不会影响到其他的客户端。
 - 可维护性: 更容易维护和升级代码，因为可以独立地修改和扩展接口。

(I) 接口分离原则

- 接口中不包含客户端不使用的方法
 - 接口应该小而精确；
 - 接口不应该提供客户端不需要的方法；
 - 因为模块需要实现这些方法
 - 避免胖接口
- 常见问题
 - 客户需要——接口
 - 自己的实现——接口

严禁复制

bad example: 不需要的接口也提供



例1：业务故事

01

“多媒体”接口,包含了播放、录制、编辑。

音频播放器:播放。

录音机:录制。

视频播放器:播放和录制。

视频剪辑:编辑和播放。

02

拆分成更细粒度的接口:

播放接口、

录制接口

编辑接口。

严禁复制

1: 违反ISP

MediaPlayer和
VideoPlayer
被迫实现同一个接口

```
interface MultimediaDevice {  
    void play();  
    void record();  
    void edit();  
    void adjustVolume();  
    void setSpeed();  
    // ... 更多方法  
}  
  
class MediaPlayer implements Multimedia {  
    public void play() { }  
    public void record() {  
        throw new UnsupportedOperationException("");  
    }  
}
```

2 符合ISP

```
interface Playable {  
    void play();  
}
```

```
interface Recordable {  
    void record();  
}
```

```
class MediaPlayer implements Playable {  
    public void play() { ... }  
}
```

```
class AudioRecorder implements  
Recordable {  
    public void record() { ... }  
}
```

分拆多个接口，根据自己的需要进行选择

禁止转载

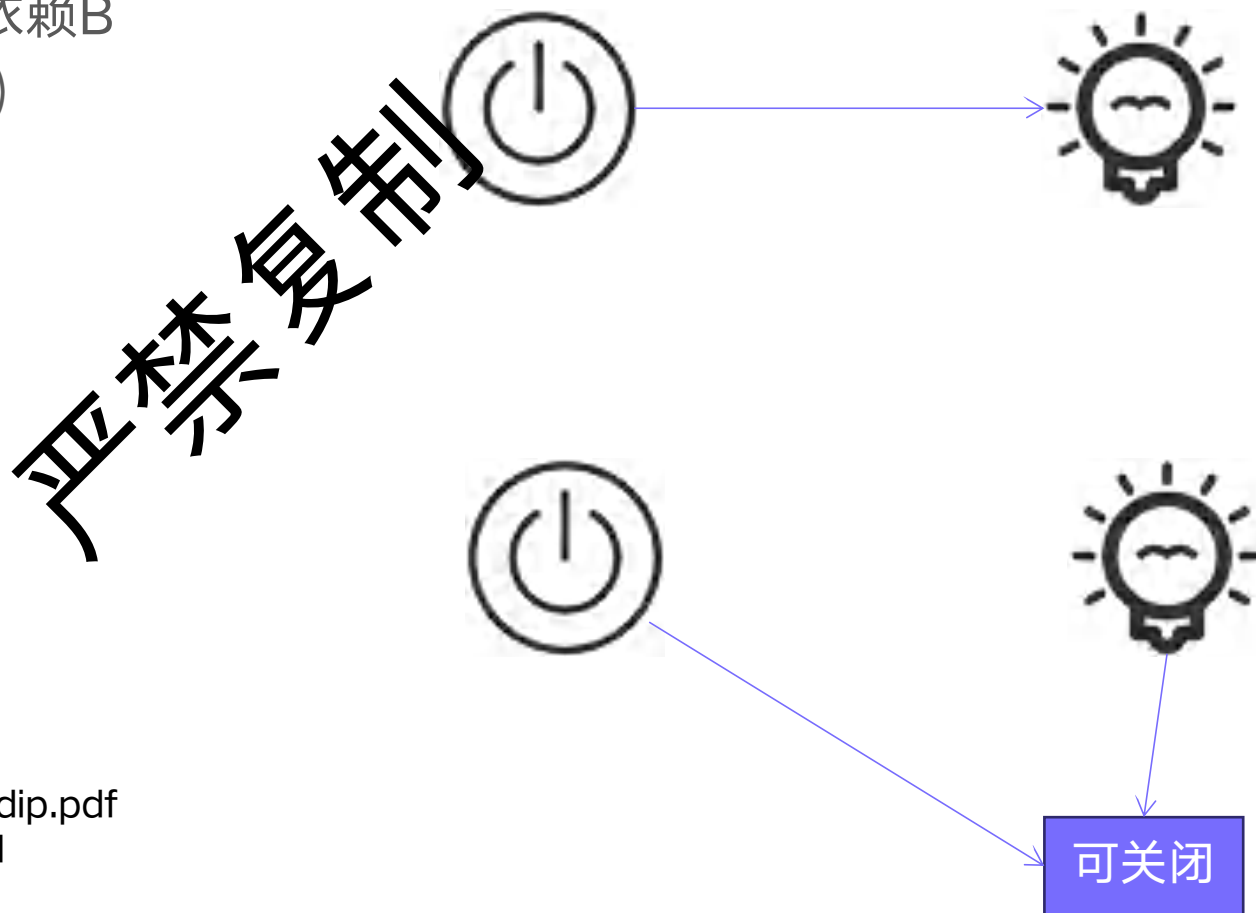
```
class VideoPlayer implements Playable, Recordable {  
    public void play() { ... }  
    public void record() { ... }  
}
```

依赖反转原则 (Dependency Inversion Principle, DIP)

- 定义:
 - 高层模块不应该依赖于低层模块。两者都应该依赖于抽象。
 - 抽象不应该依赖于细节。细节应该依赖于抽象。
- 含义:
 - 应该使用抽象（接口或抽象类）来解耦高层模块和低层模块，使得它们可以独立地变化。
- 好处:
 - 解耦: 减少类之间的依赖关系，降低代码的耦合性。
 - 灵活性: 可以更容易地修改和扩展模块，而不会影响到其他的模块。
 - 可测试性: 更容易对模块进行单元测试，因为可以使用 mock 对象来模拟依赖的模块。

(D) 依赖关系反转原则

- 对传统的依赖关系进行反转
 - A需要使用B，传统方式会导致A依赖B
 - A和B都依赖于一个C（是B的抽象）
 - 抽象不能依赖于实现细节
 - 实现细节应该依赖于抽象
- 可以使用的技术
 - plugin
 - 依赖注入
 - service locator



<http://www.objectmentor.com/resources/articles/dip.pdf>
<http://martinfowler.com/articles/dipInTheWild.html>

(D) 依赖关系反转原则

- 降低耦合
 - A和B的依赖关系被解除了
- 提高可测试性
 - A和B都依赖于接口C，可以在测试将B替换成B'
- 框架支持
 - Spring
 - Guice

严禁复制



例1：实现1

```
class SMSSender {  
    public void sendSMS(String message, String recipient) { }  
}  
class NotificationService {  
    private EmailSender emailSender;  
    private SMSSender smsSender;  
    public void sendNotification(String message, String recipient, String type) {  
        if ("email".equals(type)) {  
            emailSender.sendEmail(message, recipient);  
        } else if ("sms".equals(type)) {  
            smsSender.sendSMS(message, recipient);  
        }  
    }  
}
```

NotificationService 依赖于email和sms

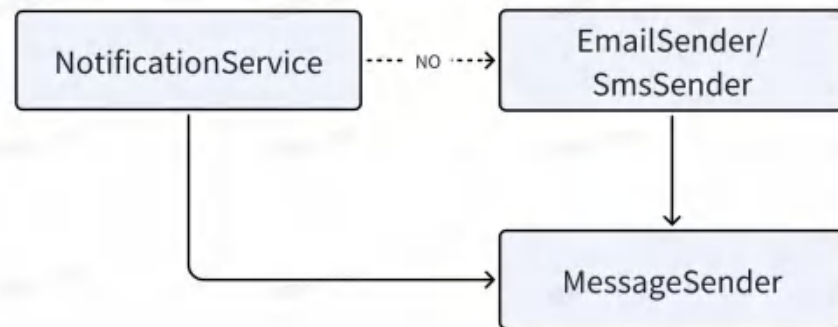
例1 实现2

// EmailSender, SMSSender依赖于MessageSender

```
class NotificationService {  
    private MessageSender messageSender;  
    // 依赖于MessageSender 接口  
    public void sendNotification(String message, String recipient) {  
        messageSender.sendMessage(message, recipient);  
    }  
}
```

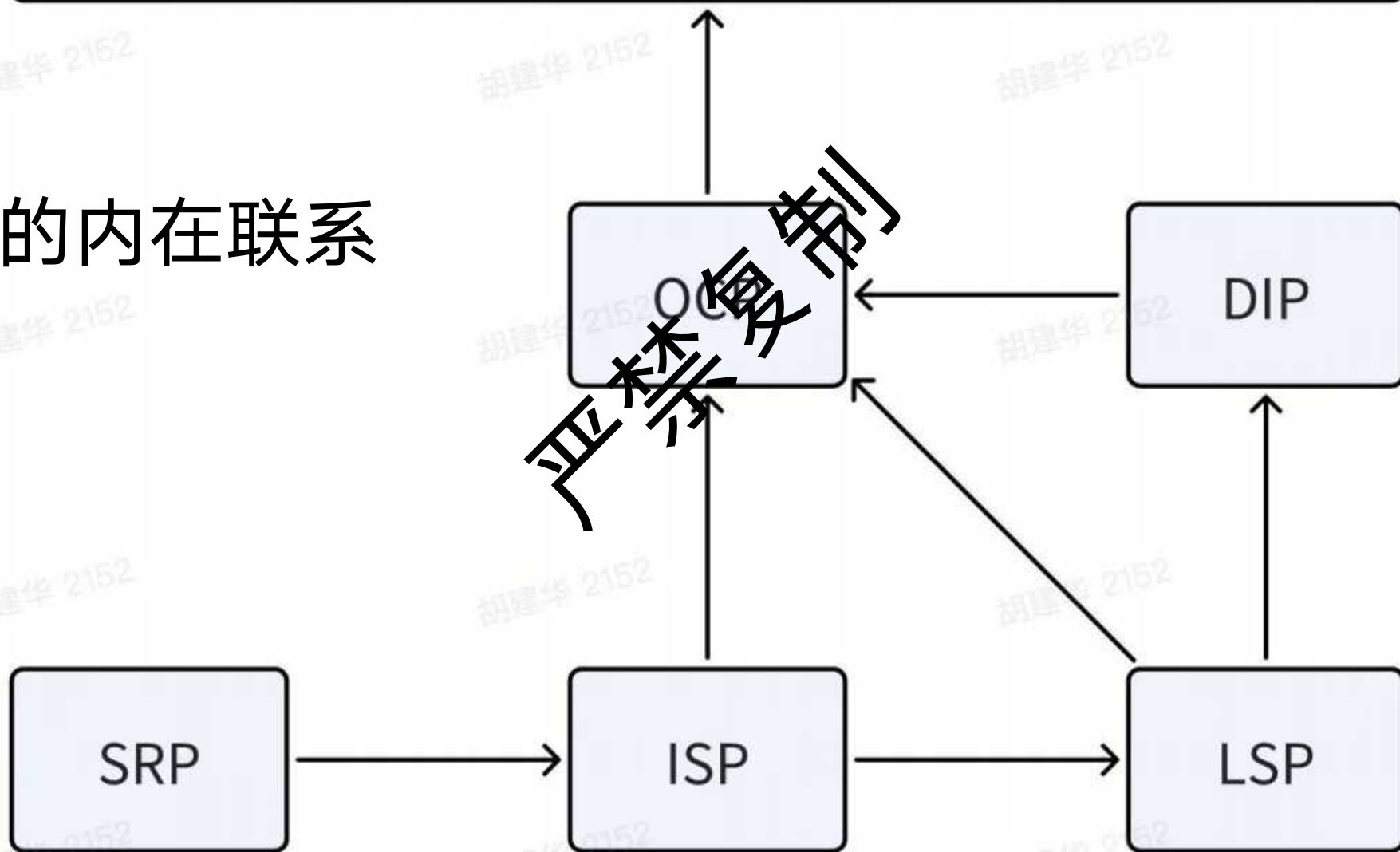
// 应用侧，实现了NotificationService 与email的解耦合，更加灵活

```
MessageSender emailSender = new EmailSender();  
NotificationService emailNotification = new NotificationService(emailSender);  
emailNotification.sendNotification("您的订单已发货", "user@example.com");
```



可扩展，可维护，可复用的，高内聚，低耦合的系统

SOLID的内在联系





提高内聚

第五章

严禁复制



提高内聚性

- 内聚性：模块内部的元素之间协作的紧密程度
 - 内部元素：class, function, line of code
- 高内聚是好的
 - 每个模块必须具备一项能力（不要纠结什么是能力）
 - 模块的粒度：可以独立的发布和复用
 - 可以对每个模块进行独立的测试

严禁复制

内聚性指标：LCOM

- LCOM（方法内聚的缺失度）

- 度量一个class的各个方法之间的相似度

- 公式：LCOM越大内聚性越差

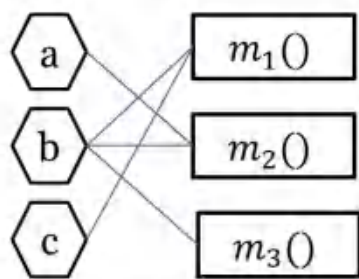
- $LCOM = P - Q$ IF $P > Q$

- $LCOM = 0$ IF $P \leq Q$

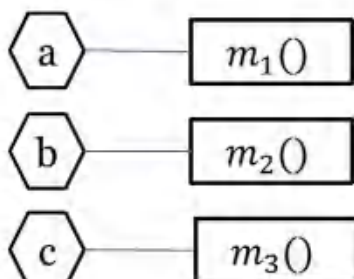
- P：与其它方法没有共同属性的方法数量

- Q：与其它方法有共同属性的方法数量

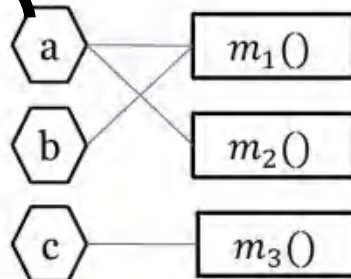
- 方法与属性：verify (userName, password)



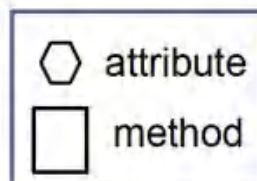
$|P| = 0, |Q| = 3$
 $LCOM = 0$



$|P| = 3, |Q| = 0$
 $LCOM = 3$



$|P| = 2, |Q| = 1$
 $LCOM = 1$



例子

```
// 类 1: 高内聚 - LCOM 值较低
class HighCohesionExample {
    private int data1;
    private int data2;

    public HighCohesionExample(int data1, int data2) {
        this.data1 = data1;
        this.data2 = data2;
    }

    public int methodA() {
        // 使用 data1 和 data2
        return data1 + data2;
    }

    public void methodB(int value) {
        // 修改 data1
        this.data1 = value;
    }

    public int methodC() {
        // 使用 data2
        return data2 * 2;
    }
}
```

```
class LowCohesionExample {
    private int dataX;
    private String dataY;
    private boolean dataZ;

    public LowCohesionExample(int dataX, String dataY, boolean dataZ) {
        this.dataX = dataX;
        this.dataY = dataY;
        this.dataZ = dataZ;
    }

    public int calculate() {
        // 只使用 dataX
        return dataX * 2;
    }

    public String format() {
        // 只使用 dataY
        return "Formatted: " + dataY;
    }

    public boolean validate() {
        // 只使用 dataZ
        return dataZ;
    }

    public void unrelatedMethod() {
        // 不使用任何实例变量
    }
}
```

提高内聚性应该遵循3原则

- REP-复用/发布等同性原则
- CCP-共同闭包原则
- CRP-共同复用原则

严禁复制

RREP 复用/发布等价原则 (release)

- RREP 表示如果两个或多个类经常一起被复用，那么它们应该被放在同一个组件中发布。
 - 换句话说，组件的粒度应该与复用的粒度一致。
- 当用户下单时，OrderService 会调用 PaymentService 来完成支付操作。这两个服务经常一起被使用。
 - Component: OrderAndPayment
 - - OrderService
 - - PaymentService

严禁复制

修改闭包原则（change）

- CCP 表示一个组件中的所有类应该共同修改以应对同一类变更。
 - 换句话说，如果某些类因为同一种原因需要修改，那么它们应该被放在同一个组件中。
- 在电商系统中，需要支持多种支付方式（如信用卡、PayPal、微信支付等）。
因为支付协议的变化而需要修改：
 - Component: PaymentMethods
 - - CreditCardPayment
 - - PayPalPayment
 - - WeChatPayment

严禁复制

共同复用原则（reuse）

- CRP 表示一个组件中的所有类都应该被一起复用。
 - 换句话说，组件中的每个类都应该对复用者有价值，否则就不应该包含在这个组件中。
- 电商系统中：
 - UserService：用户注册、登录等功能，
 - ProductService 提供了商品查询和管理的功能。。
- Component: UserManagement
 - - UserService
- Component: ProductManagement
 - - ProductService



降低耦合

第六章

严禁复制

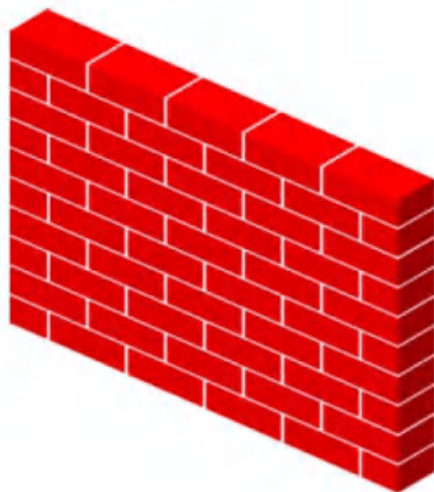


耦合

- 耦合=模块之间的互相依赖
 - 降低耦合 导致 系统更容易被修改
- 低耦合才能独立部署每个模块
 - 在其它模块发生变化时，自己才能保持稳定

严禁复制

无耦合



有耦合



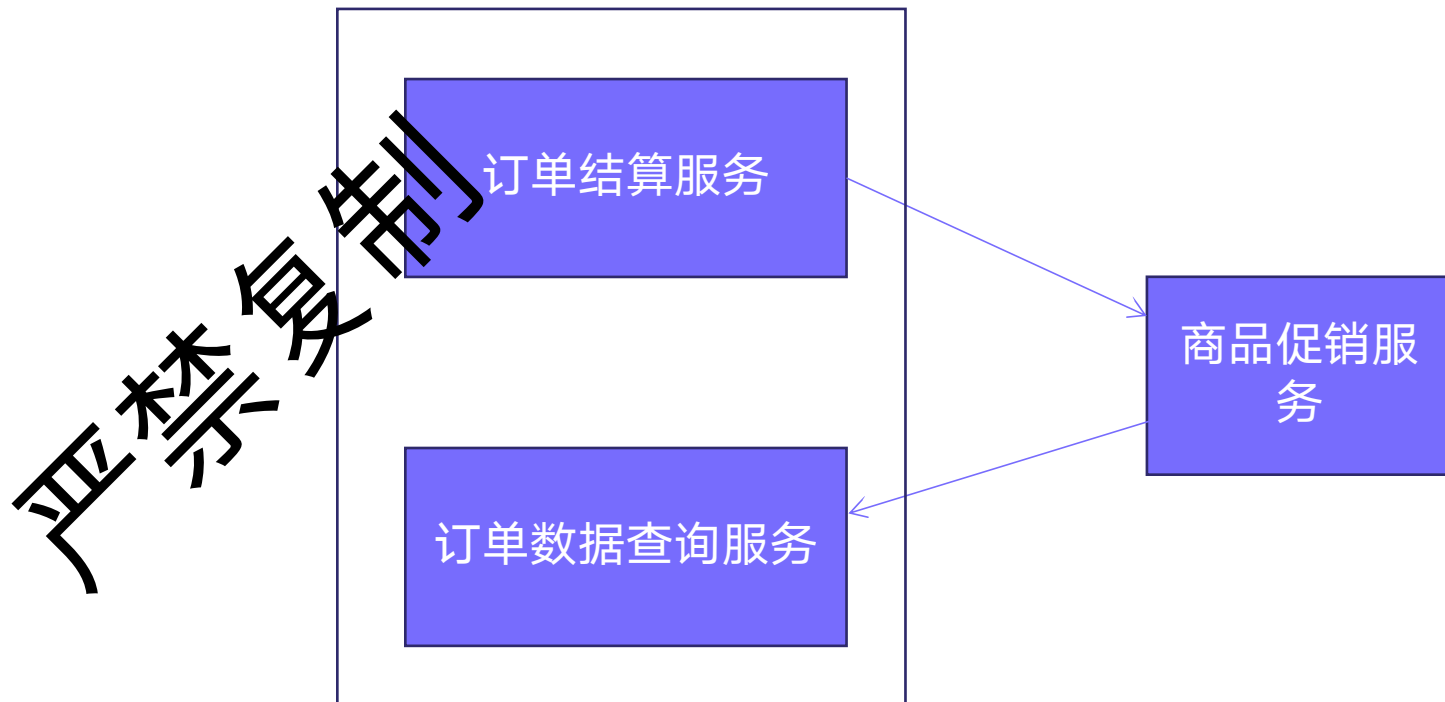
耦合的3原则

- ADP——不循环依赖原则
- SDP——稳定性依赖原则
- SAP——稳定性抽象原则
 - 稳定性：不容易改变，难以改变，是对象的固有属性，与实现无关

严禁复制

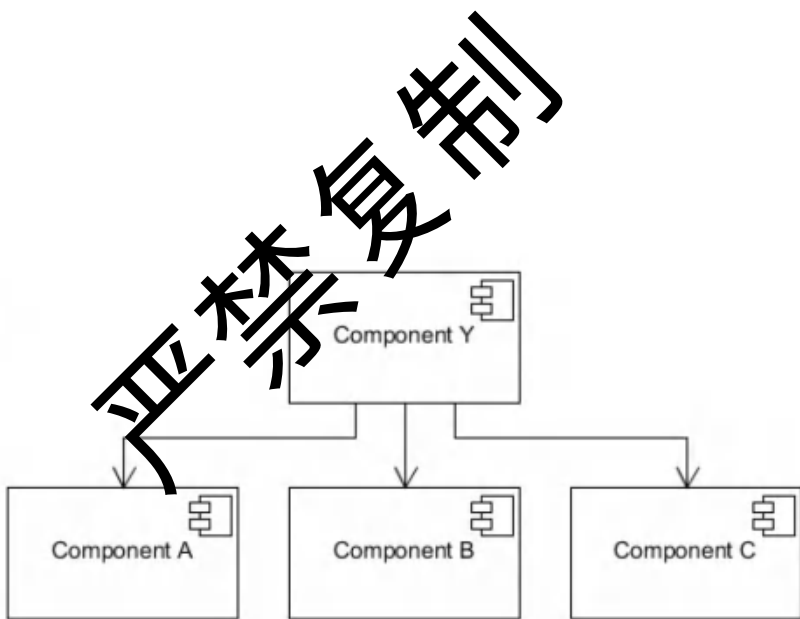
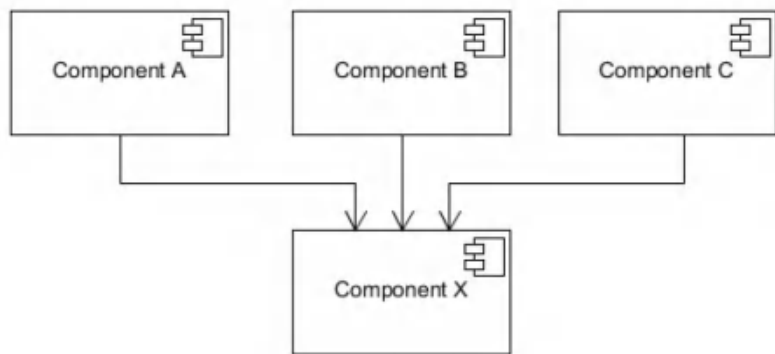
ADP-不循环依赖原则

- 模块之间的依赖关系必须构成DAG
 - 无环
 - cycle导致变化很困难
 - 很多模块被影响
 - 构建顺序很困难
 - 可以通过split解决cycle



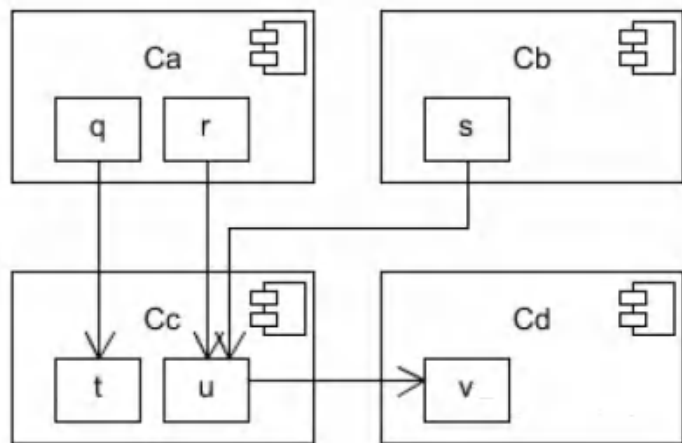
稳定依赖原则

- 组件之间的依赖关系应该是稳定性递增的
 - stable——more stable
- 变化reason越少越稳定
 - X需要稳定，Y不需要稳定



稳定性指标

- fan-in: incoming 依赖
- fan-out: outgoing 依赖
- 不稳定性 $I = (\text{fan_out}) / (\text{fan_in} + \text{fan_out})$
 - 1 不稳定, 0 稳定



$$I(Ca) = \frac{2}{0+2} = 1$$

$$I(Cb) = \frac{1}{0+1} = 1$$

$$I(Cc) = \frac{1}{3+1} = \frac{1}{4}$$

$$I(Cd) = \frac{0}{1+0} = 0$$

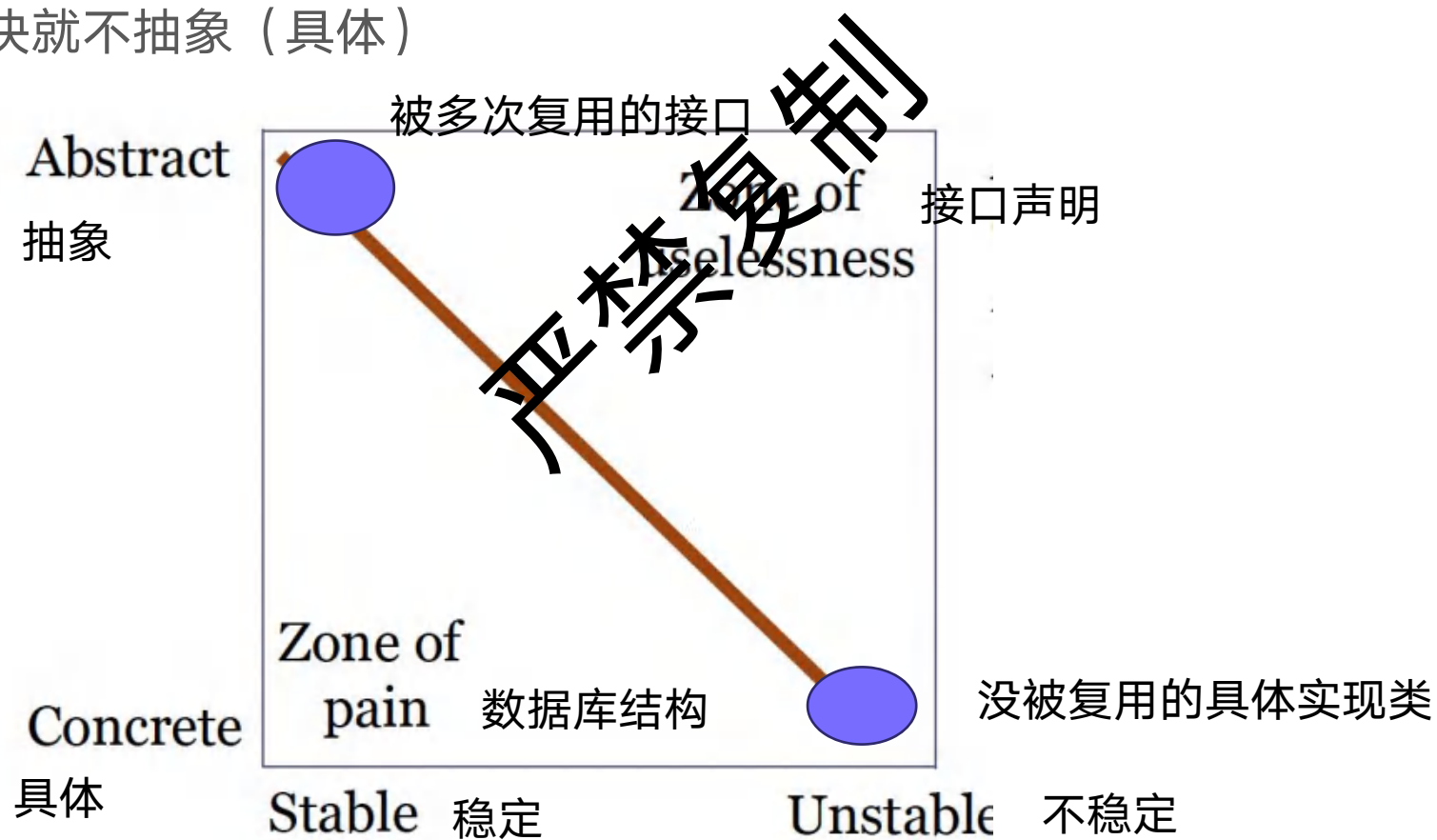
被使用的越多越稳定，依赖越多越不稳定

稳定抽象原则

- 一个模块的 稳定性 正比于 抽象性
 - 最稳定的模块应该最抽象
 - 不稳定的模块就不抽象（具体）

- bad:

- 多次复用
- 具体实现



3种不同的抽象层次

```
public interface Product {  
    String getId();  
    String getName();  
    double getPrice();  
    double getFinalPrice();  
    // 最终价格，可能包括折扣等  
}
```

```
public abstract class ProductModel  
implements Product {  
    private String id;  
    private String name;  
    private double price;  
    public abstract double  
    getFinalPrice(); // 抽象方法，由子类实  
    现  
}
```

```
public class ConcreteProduct extends  
ProductModel {  
    private int stock;  
    public double getFinalPrice() {  
        // 假设有一个全局折扣规则  
        double discountRate = 0.1; //  
        10% 折扣  
        return super.getPrice() * (1 -  
        discountRate);  
    }  
}
```



共生代码

第七章

严禁复制



共生



共生现象

- 共生性：当两个元素必须一起更改以使软件保持正常工作时
 - 加密 解密
- 改变一个就必须改变另一个
 - 意味着修改困难（不知道另一个在哪里）
 - 是一种耦合
 - 把内聚转变成耦合
- 共生的类型
 - 静态 = 可以通过静态代码分析发现
 - 动态 = 通过运行时发现

严禁复制

静态共生

- name
 - 多个element对实体name达成一致
- type
 - 多个element对实体类型达成一致（或结构）
- meaning含义
 - 多个element对某个数字的含义达成一致（魔法数字，MaxBufSize）
- position调用顺序
 - 多个element对多个value或动作的顺序达成一致（同类型的多个参数）
- 算法
 - 多个模块必须使用同一算法（加密 解密）

```
public class Time {  
    int _hour; int _min; int _sec;  
    public Time(int hour, int min, int sec) {  
        _hour = hour ;  
        _minute = minute ;  
        _second = second ;  
    }  
    public String display() {  
        return _hour + ":" + _min + ":" + _sec ;  
    }  
}  
public class Client {  
    val noon = Time(12,0,0);  
    ...  
}
```

动态共生

- 运行

- 不同element对多个步骤的**执行顺序**达成一致

- 时机

- 不同element对多个步骤**执行时机**达成一致（A在9点执行，B在10点执行）

- 值约束

- ……对多个value的约束达成一致（ $A < B < C$ ）

- 身份共享

- 多个对象必须引用同一个身份对象（信息，权限，行动-》ID）

```
Email email = new Email();  
email.setRecipient("foo@example.comp");  
email.setSender("me@mydomain.com");  
email.send();  
email.setSubject("Hello World");
```

严禁复制

共生的3个属性

- Degree宽度
 - 多少元素参与
- Locality距离
 - 同一个函数，同一个类，同一个模块，跨模块
- Strength强度
 - 解除的难度

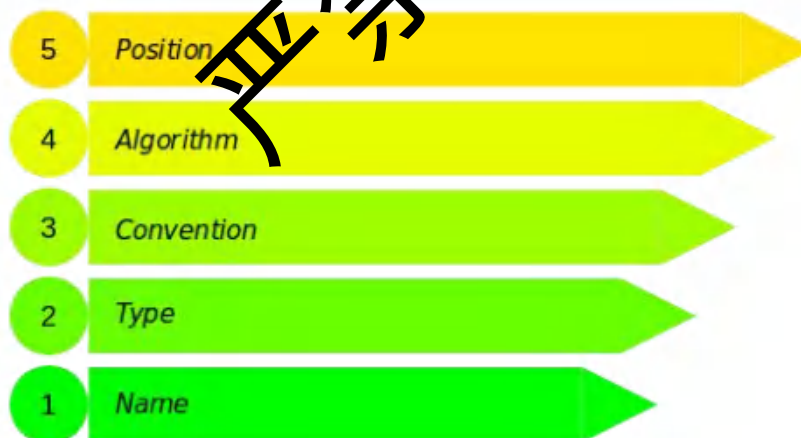
严禁复制

不同类型的“共生”的强度

Dynamic:



Static:



如何降低共生

- 共生对模块化的影响
 - 增加耦合
 - 增加维护难度
- 从3个方向进行重构
 - 最小化距离
 - 放到一个类里面
 - 最小化元素个数
 - 复用同一个函数
 - 最小化强度
 - 把动态共生变成静态共生

方法1{
.....
String key= “.....” + “.....”
redis.put(key,value)
.....
方法2{
.....
String key= “.....” + “.....”
redis.get(key,value)
.....
}



模块集成

第八章

严禁复制



模块的集成

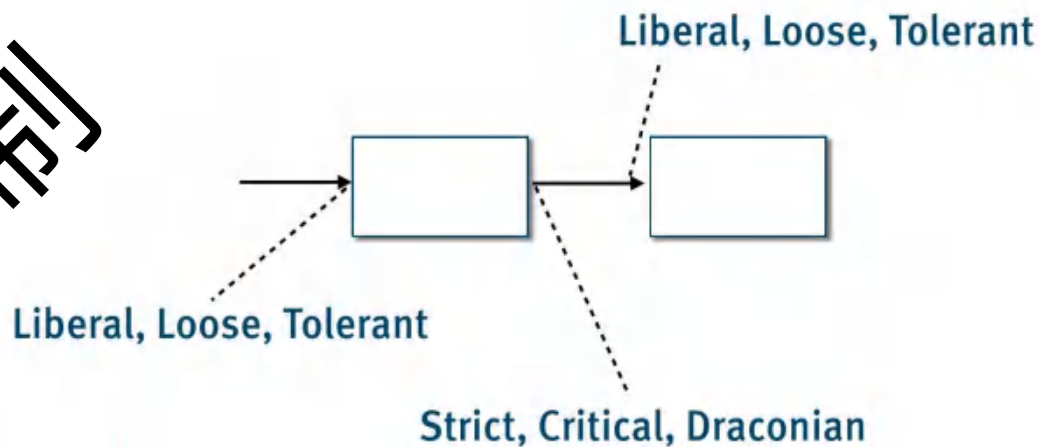
- Postel法则
- Demeter法则
- 流畅的API模式

严禁复制

健壮性原则，Postel法则

- Postel法则，1980 TCP/IP
 - 对模块接收的保持开放态度，对模块send的保守态度
 - 严于律己宽以待人
- 提高互操作性（易于集成）
 - 发送的message要结构良好
 - 能够accept包含错误的消息
- 法则的应用
 - 只处理自己感兴趣的fields，忽略其它fields
 - 允许其它API发生变化（不影响自己）

严禁复制



<https://devopedia.org/postel-s-law>

代码例子

```
public String parseDate(String input) {  
    static {  
        DATE_FORMATS.add("yyyy/MM/dd"); // 2023/1/15  
        DATE_FORMATS.add("dd-MM-yyyy"); // 15-1-2023  
        DATE_FORMATS.add("MMMM d, yyyy"); // January 15, 2023  
        DATE_FORMATS.add("yyyy-MM-dd"); // 2023-01-15  
    }  
  
    for (String format : DATE_FORMATS) {  
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern(format);  
  
        LocalDate date = LocalDate.parse(input, formatter);  
        // 输出标准格式 YYYY-MM-DD  
  
        return date.format(DateTimeFormatter.ISO_LOCAL_DATE);  
    }  
}
```

Demeter法则

- 少量知识原则
- Demeter 系统
 - 一个units只对另外一个units持有有限的知识
 - 只有紧密关联的units
 - 每个单元只跟他的朋友交谈
 - 不与陌生人讲话
 - 不合理的设计
 - a.b.c.method()
 - 调用者是a的朋友，但是不是c的朋友，不应该访问c的方法
- 意义
 - 降低耦合
 - 有时候难以遵守

```
class Person {  
    private Car car;  
  
    public Person(Car car) {  
        this.car = car;  
    }  
  
    public void startCarEngine() {  
        // 违反Demeter法则：直接访问  
        Car的Engine  
        car.getEngine().start();  
    }  
}
```

Fluent APIs 流畅的API

- 一种API设计方式
- 提升接口的可读性、易用性
- 优势
 - 创造出领域特定的语言
 - 让IDE（LLM）帮你写代码
 - 并不违背Demeter法则

```
Result<Record3<Integer, String, String>> result = create
    .select(USERS.ID, USERS.USERNAME, )
    .from(USERS)
    .where(USERS.AGE.gt(18))
    .and(USERS.USERNAME.like("%john%"))
    .orderBy(USERS.ID.asc())
    .limit(10)
    .fetch();
```

禁止复制

GRASP模式

- GRASP: 面向对象设计的责任分配模式
 - 定义: General Responsibility Assignment Software Patterns
 - 目的: 指导如何在面向对象设计中分配责任
 - 提出者: Craig Larman
- 应用GRASP的价值:
 - 提供系统化的面向对象设计方法
 - 帮助合理分配责任
 - 创建高质量、可维护的系统
 - 培养良好的设计思维

严禁复制

原则详解（1-5）

1. 信息专家: 将职责分配给拥有完成该职责所需信息的类

- 例: OrderItem计算自己的价格

2. 创建者: 负责创建对象的类应该是:

- 包含或聚合该对象的类, 记录该对象的类, 紧密使用该对象的类, 拥有初始化数据的类

3. 控制器: 负责接收和协调系统事件的非UI对象

- 系统控制器或用例控制器

4. 低耦合: 减少类之间的依赖关系

- 提高可维护性和灵活性

5. 高内聚: 确保类的职责紧密相关, 专注于单一目的

- 避免"上帝类"现象

原则详解（6-9）

6. 多态: 使用多态而非条件语句处理类型变化

- 支持开闭原则

7. 纯虚构: 创建不代表问题域概念的服务类

- 例: 数据访问对象(DAO)、工具类

8. 间接性: 通过中间对象减少直接耦合

- 应用: 适配器、代理、中介者模式

9. 防止变异: 创建稳定接口隔离变化点

- "封装变化"的核心思想

其它模块化战术

- 提升模块的外部配置
- 提供接口的缺省实现
- DRY (Dont repeat yourself)
 - 不要重复编写
- YAGNI
 - 你不需要它
- KISS
 - 做最简单的实现

严禁复制